

NAME : P SAI VENKAT RAJU

SUBJECT : AI ASSISTED CODING

LAB TEST – 3

ROLL NO : 2403A510G0

Set E11

Q1:

Scenario: In the Retail sector, a company faces a challenge related to data structures with ai.

Task: Use AI-assisted tools to solve a problem involving data structures with ai in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output

PROMPT USED :

Write a minimal python script that analyses a small retail dataset and used product dict and transactions list as input and explains the ai assistance used

CODE GENERATED :

```
1 # ...existing code...
2 import itertools
3 from collections import Counter
4 import datetime
5 import math
6
7 PRODUCTS = {
8     "P001": {"name": "Basic T-Shirt", "inventory": 50, "reorder_point": 20},
9     "P002": {"name": "Jeans", "inventory": 30, "reorder_point": 10},
10    "P003": {"name": "Sneakers", "inventory": 15, "reorder_point": 8},
11 }
12
13 TRANSACTIONS = [
14     ("2025-10-01", ["P001", "P002"]),
15     ("2025-10-01", ["P001"]),
16     ("2025-10-02", ["P003"]),
17     ("2025-10-02", ["P001", "P003"]),
18     ("2025-10-03", ["P001", "P002"]),
19     ("2025-10-04", ["P002", "P003"]),
20     ("2025-10-05", ["P001"]),
21     ("2025-10-06", ["P002"]),
22     ("2025-10-07", ["P001", "P003"]),
23 ]
24 # ...existing code...
25 def supports(tx):
26     n=len(tx);c=Counter()
27     for _,items in tx: c.update(items)
28     return {k:c[k]/n for k in c},c
29
30 def pairs(tx,min_s=0.2):
31     n=len(tx);pc=Counter()
32     for _,items in tx: pc.update(itertools.combinations(sorted(set(items)),2))
33     return {p:cnt/n for p, cnt in pc.items() if cnt/n>=min_s},pc
```

Output:

```
P003 Sneakers: support=0.44, sold=4
Pairs:
Velocity:
P001 Basic T-Shirt: 0.86/day
P002 Jeans: 0.57/day
P003 Sneakers: 0.57/day
Restock:
None
```

Explanation of AI assistance used

- An LLM was used to simplify the original solution: It recommended a very small, easy-to-read approach (single-item supports, naive pair counting, simple 7-day velocity restock).
- The model translated that design into minimal Python code using only the standard library, and suggested clear, short printing for sample output.

Sample output (when run)

lab3.py Current file

Ask Copilot

A... GPT-5

```
20 ("2025-10-05", ["P001"]),
21 ("2025-10-06", ["P002"]),
22 ("2025-10-07", ["P001", "P003"]),
23
24 ...existing code...
25 def supports(tx):
26     n=len(tx);c=Counter()
27     for _,items in tx: c.update(items)
28     return {k:c[k]/n for k in c},c
29 def pairs(tx,min_s=0.2):
30     n=len(tx);pc=Counter()
31     for _,items in tx: pc.update(itertools.combinations(sorted(set(items)),2))
32     return {p:cnt/n for p, cnt in pc.items() if cnt/n>=min_s},pc
33 def vel_rest(tx):
34     dates=[datetime.date.fromisoformat(d) for d,_ in tx];days=(max(dates)-min(dates)).days+1;_c=supports(tx)
35     v={p:c.get(p,0)/max(days,1) for p in PRODUCTS}
36     r=[(p,meta["name"],meta["inventory"],max(math.ceil(v.get(p,0)*7+meta["reorder_point"]-meta["inventory"]),0)) for p,meta in PRODUCTS]
37     return v,c,days,r
38 f __name__ == "__main__":
39     sup,cnt=supports(TRANSACTIONS);pr,pc=pairs(TRANSACTIONS,0.25);vel,cnt,sugg=vel_rest(TRANSACTIONS)
40     print("Singles:");print(f" {p} {PRODUCTS[p]['name']}: support={s:.2f}, sold={cnt.get(p,0)}" for p,s in sorted(sup.items(),key=lambda p:s[supports(p)]))
41     print("Pairs:");print(f" {p} {PRODUCTS[p]['name']}: support={s:.2f} (count={pc[p]})" for p,s in sorted(pr.items(),key=lambda p:s[supports(p)]))
42     print("Velocity:");print(f" {p} {PRODUCTS[p]['name']}: {vel[p]:.2f}/day" for p in vel)
43     print("Restock:");print(" None" if not sugg else "")[[print(f" {p} {n}: inv={inv}, qty={qty}" for p,n,inv,qty in sugg)]
```

OUTPUT :

```
P003 Sneakers: support=0.44, sold=4
Pairs:
Velocity:
P001 Basic T-Shirt: 0.86/day
P002 Jeans: 0.57/day
P003 Sneakers: 0.57/day
Restock:
None
```

```
P003 Sneakers: support=0.44, sold=4
Pairs:
Velocity:
P001 Basic T-Shirt: 0.86/day
P002 Jeans: 0.57/day
P003 Sneakers: 0.57/day
Restock:
None
```

OBSERVATIONS :

An LLM was used to simplify the original solution: it recommended a very small, easy-to-read approach (single-item supports, naive pair counting, simple 7-day velocity restock).

The model translated that design into minimal Python code using only the standard library, and suggested clear, short printing for sample output

Q2:

Scenario: In the E-commerce sector, a company faces a challenge related to data

structures

with ai.

Task: Use AI-assisted tools to solve a problem involving data structures with ai in this context.

Deliverables: Submit the source code, explanation of AI assistance used, and sample output.

PROMPT USED :

Write a single file python script with standard library only that analyses a e commerce dataset with requirements of PRODUCTS,ORDERS ON DATASETS and to print item supports and per user recommendations and keep the code concise and readable (aim<=20 lines) handle empty profiles safely

CODE GENERATED :

```
1 # ...existing code...
2 import datetime
3 from collections import Counter
4
5 PRODUCTS = {
6     "A": {"name": "Phone Case"},
7     "B": {"name": "Wireless Earbuds"},
8     "C": {"name": "Portable Charger"},
9 }
10
11 ORDERS = [
12     ("user1", "2025-10-01", ["A", "B"]),
13     ("user2", "2025-10-01", ["A"]),
14     ("user3", "2025-10-02", ["B", "C"]),
15     ("user1", "2025-10-03", ["C"]),
16     ("user2", "2025-10-04", ["B", "C"]),
17 ]
18
19 def item_supports(orders):
20     n = len(orders)
21     c = Counter(pid for _, _, items in orders for pid in items)
22     return {pid: c[pid] / n for pid in c}, c
23
24 def user_profiles(orders):
25     profiles = {}
26     for user, _, items in orders:
27         profiles.setdefault(user, set()).update(items)
28     return profiles
29
30 def recommend(user, profiles, top_n=2):
31     target = profiles.get(user, set())
32     scores = Counter()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

pranav@PRANAVs-MacBook-Air AI ASSISTED CODING % cd /Users/Shared/AI\ ASSISTED\ CODING ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/as-python-debugpy-2025.14.1-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 57401 -- /Users/Shared/AI\ ASSISTED\ CODING/lab3.py

Item support (fraction of orders):
A Phone Case: 0.40 (count=2)
B Wireless Earbuds: 0.60 (count=3)
C Portable Charger: 0.60 (count=3)

User profiles and recommendations:
user1: items=['A', 'B', 'C'] -> recommends=[]
user2: items=['A', 'B', 'C'] -> recommends=[]
user3: items=['B', 'C'] -> recommends=['A']

pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %

(IDs e.g. "A","B","C") and Of dict, Counter), user_profiles ions in a short, clear __mair 0 lines), handle empty profil al packages.

Explanation of AI assistance (unique)

- The AI was used as a development assistant to design a compact solution and to iteratively compress the implementation while retaining clarity.
- It suggested appropriate Python primitives (Counter, set operations) and a simple similarity metric (Jaccard) that map directly to the e-commerce problem (co-purchase recommendation).
- It performed micro-refactoring: extracting small functions, removing redundancy, and producing a short demo block so the script is ready to run.
- The AI also proposed minimal input data and concise printed output so results are easy to inspect for grading or quick validation.

Example output (illustrative)

Add Context...

lab3.py Current file

A... GPT-5 ...

```
def user_profiles(orders):
    profiles.setdefault(user, set()).update(items)
    return profiles

def recommend(user, profiles, top_n=2):
    target = profiles.get(user, set())
    scores = Counter()
    for other, items in profiles.items():
        if other == user: continue
        inter = target & items
        uni = target | items
        sim = (len(inter) / len(uni)) if uni else 0
        for it in items - target:
            scores[it] += sim
    return [pid for pid, _ in scores.most_common(top_n)]

if __name__ == "__main__":
    sup, counts = item_supports(ORDERS)
    profiles = user_profiles(ORDERS)
    print("Item support (fraction of orders):")
    for pid, s in sup.items(): print(f"{pid} {PRODUCTS[pid]['name']}: {s:.2f} (count={counts[pid]})")
    print("\nUser profiles and recommendations:")
    for u in profiles:
        rec = recommend(u, profiles)
        print(f"{u}: items={sorted(profiles[u])} -> recommend={rec}")
```

Item support (fraction of orders):
A Phone Case: 0.40 (count=2)
B Wireless Earbuds: 0.60 (count=3)
C Portable Charger: 0.60 (count=3)

User profiles and recommendations:
user1: items=['A', 'B', 'C'] -> recommend=[]
user2: items=['A', 'B', 'C'] -> recommend=[]
user3: items=['B', 'C'] -> recommend=['A']

OUTPUT :

```
Item support (fraction of orders):
A Phone Case: 0.40 (count=2)
B Wireless Earbuds: 0.60 (count=3)
C Portable Charger: 0.60 (count=3)

User profiles and recommendations:
user1: items=['A', 'B', 'C'] -> recommend=[]
user2: items=['A', 'B', 'C'] -> recommend=[]
user3: items=['B', 'C'] -> recommend=['A']
```

OBSERVATIONS MADE :

The AI was used as a development assistant to design a compact solution and to iteratively compress the implementation while retaining clarity.

It suggested appropriate Python primitives (Counter, set operations) and a simple similarity metric (Jaccard) that map directly to the e-commerce problem (co-purchase recommendation).

It performed micro-refactoring: extracting small functions, removing redundancy, and producing a short demo block so the script is ready to run.